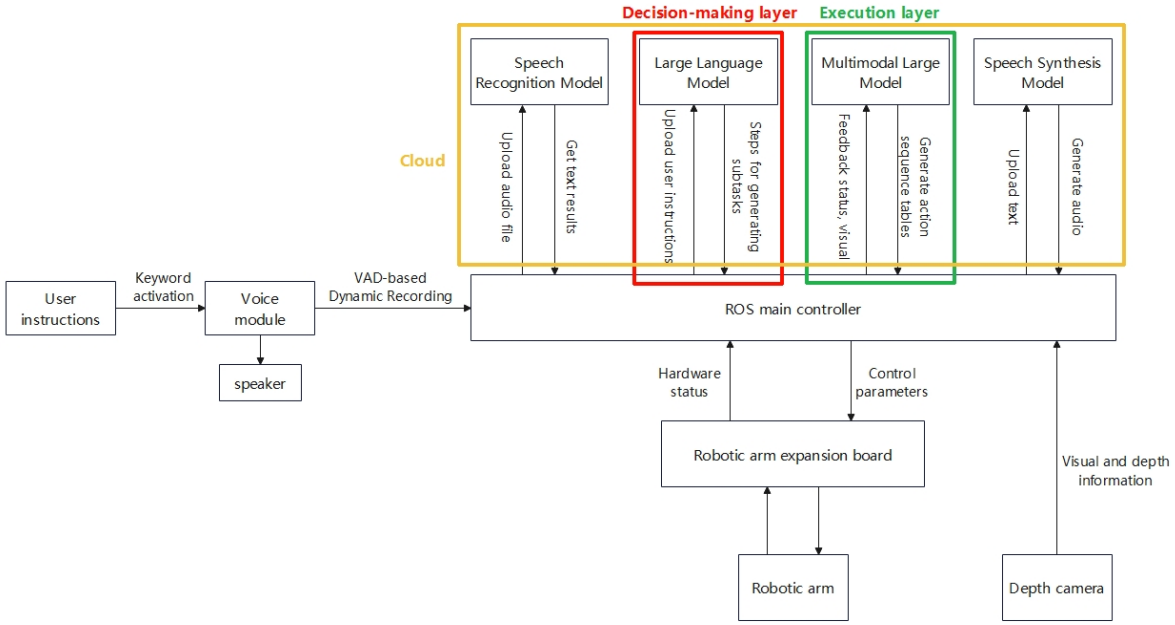
Embodied Intelligent Robot System Architecture

1. Course Content

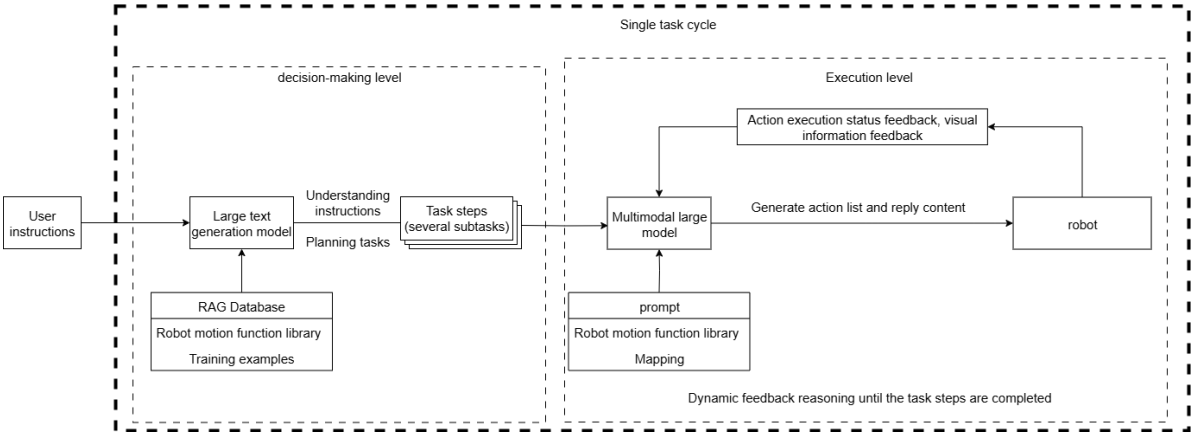
1. Explanation of the large model inference architecture for AI embodied intelligent robots
2. Introduction to basic concepts in the system to prepare for subsequent practical courses

2. System Composition



3. AI Large Model Inference Architecture Diagram

The robot adopts a **dual-model inference** and **dynamic feedback inference** design in program architecture, improving the robot's processing capability when facing long-process complex tasks. Compared to single-model architectures, it possesses stronger system robustness. Here are several key concepts: decision-making layer large model, execution layer large model, task cycle, conversation history, which will be explained individually below.



4. Large Model Inference Architecture Explanation

4.1 Dual-Model Architecture Principles

- The robot's large model control system is designed based on text generation large models and multimodal large models, where the text generation large model serves as the **decision-making layer**, functioning as an upper-level task planner to decompose complex and abstract human instructions into tasks
- The multimodal large model serves as the execution layer, responsible for receiving task steps generated by the **decision-making layer** and users' **temporary instructions** (generally simple instructions requiring the robot to end tasks, rest), supervising the progress of robot task execution and making real-time adjustments to execution actions, generating action function lists that the robot can parse and replies to users
- The action function list contains pre-written basic action functions that can directly output parameters controlling robot movement.

4.2 Advantages of Dual-Model Inference Architecture

4.2.1 Decoupling of Decision Logic and Action Control

- Multimodal large models are inferior to text generation large models in natural language semantic understanding and logical reasoning capabilities, and are prone to modal interference when executing complex tasks. Therefore, the system design decouples decision logic from action control.
- Text generation large models focus on semantic understanding and task planning (such as parsing "Can you help me get the blue block from room A?" and breaking down task steps into "Record current position → Navigate to room A → Get current robot perspective image → Locate blue block coordinates → Grab blue block → Return to starting position → Put down blue block"), avoiding the complexity of traditional single models needing to process action details simultaneously during semantic parsing.
- Multimodal large models are responsible for action generation and environment interaction (such as obtaining visual images to locate object coordinates, outputting action functions and parameters), reducing action logic confusion and large deviations caused by task complexity in single models.

4.2.2 Reducing Model Training Sample Complexity

When using single model reasoning, multimodal large models need to simultaneously perform "natural language understanding + environment perception + process decomposition + action function output", which easily leads to **modal interference** (language ambiguity or overly long instructions causing understanding errors). The dual-model approach uses phased processing, allowing the decision-making layer to focus on language space mapping and the execution layer to focus on vision-motion space mapping.

4.2.3 Flexible Extension

The decision-making layer and execution layer large models can be flexibly combined from various large models on the Alibaba Bailian large model platform (sometimes also called the Tongyi Qianwen platform). Through custom training samples and RAG knowledge bases, the models can adapt to tasks in different scenarios, greatly enhancing the generalization capability of AI embodied intelligent robots in different scenarios.

4.3 Decision-Making Layer Large Model

4.3.1 Role of Decision-Making Layer Model

- The robot's decision-making layer large model uses the Tongyi Qianwen-Max series model by default.
- The decision-making layer large model is mainly responsible for task planning. It can understand complex human instructions and decompose them into specific task steps.
- Each task step corresponds to the minimum action that the robot can complete, and then the execution layer large model implements specific robot movements by calling API functions from this robot's action function library.

4.3.2 Robot Action Function Library (Simplified)

The robot action library defines all the minimum actions that the robot can actually execute. When planning task steps, the decision-making layer model will select appropriate actions from this library for arrangement and combination.

Basic Action Class

- Turn on red light
- Turn on green light
- Turn on blue light
- Turn on alarm
- Turn off alarm
- Turn off light

Robotic Arm Class

- Adjust servo number x to y degrees

  Adjust the specified numbered servo to rotate to the specified angle
- Robotic arm movement
  Description: Move the robotic arm end in the specified direction for a specified distance
- Robotic arm upward movement
- Robotic arm nod
- Robotic arm shake head
- Robotic arm dance
  Similar semantics: shake head, nod gesture.
- Robotic arm applaud
  Similar semantics: applaud, applause gesture.
- Robotic arm clamp, grasp object
  Similar semantics: pick up, pick up
- Robotic arm put down item
  Description: Robotic arm releases the clamped item
- Robotic arm gripper open
  Description: Adjust servo number 6 angle to 0 degrees
- Robotic arm gripper close
  Description: Adjust servo number 6 angle to 180 degrees
- Sort, clamp machine code number x
  Description: Sort and clamp the specified numbered machine code.

- Track object, Description: Track the specified object
- Remove machine code at specified height
- Remove color blocks at specified height, Description: Automatically remove specified color blocks with height exceeding x cm, color values: 'red', 'green', 'blue', 'yellow'
- Sort x type of garbage
  Description: Call garbage sorting function, sort x type of garbage, x values are 'rec', 'tox', 'wet' or 'dry'.
- Move A to side of B
  Description: Change the spatial position of clamped object A to the left side of object B, Parameters: (x1,y1) are the pixel coordinates of the top-left corner of object A's outer bounding box, (x2,y2) are the pixel coordinates of the bottom-right corner of object A's outer bounding box, (x3,y3) are the pixel coordinates of the top-left corner of object B's outer bounding box, (x4,y4) are the pixel coordinates of the bottom-right corner of object B's outer bounding box, src represents the name of object A, if A is a color block, src represents the color of the color block, tar represents the name of object B, similarly, if B is a color block, tar represents the color of the color block; side is one of five directions: front, back, left, right, top, with values 1 representing front, 2 representing back, 3 representing left, 4 representing right, 5 representing top.
  Similar semantics include: clamp A to side of B, put A to side of B, place A to side of B
- Color block stacking
  Description: Clamp the color blocks in the image according to the given order.
- Put color blocks back to original position
  Description: Put color blocks back to their original position
  Similar semantics include: return color blocks to position, put color blocks back to original position.
- Track object
- Track machine code
- Cancel tracking machine code
- Cancel tracking object
- Remember current position of object
- Robotic arm pointing
- Garbage sorting
- Garbage cleaning
- Sort color blocks according to given sorting order

Image Acquisition Class

- Get current perspective image
  Description: Get current perspective image, used for observing environment or locating items.
- Record x seconds video
  Description: Record video of specified duration, x represents duration, unit is seconds.
- Play a guess object/small ball game
  Description: Record 20 seconds of video.
- Video understanding
  Description: Parse the content of the recorded 20-second video

Other Class

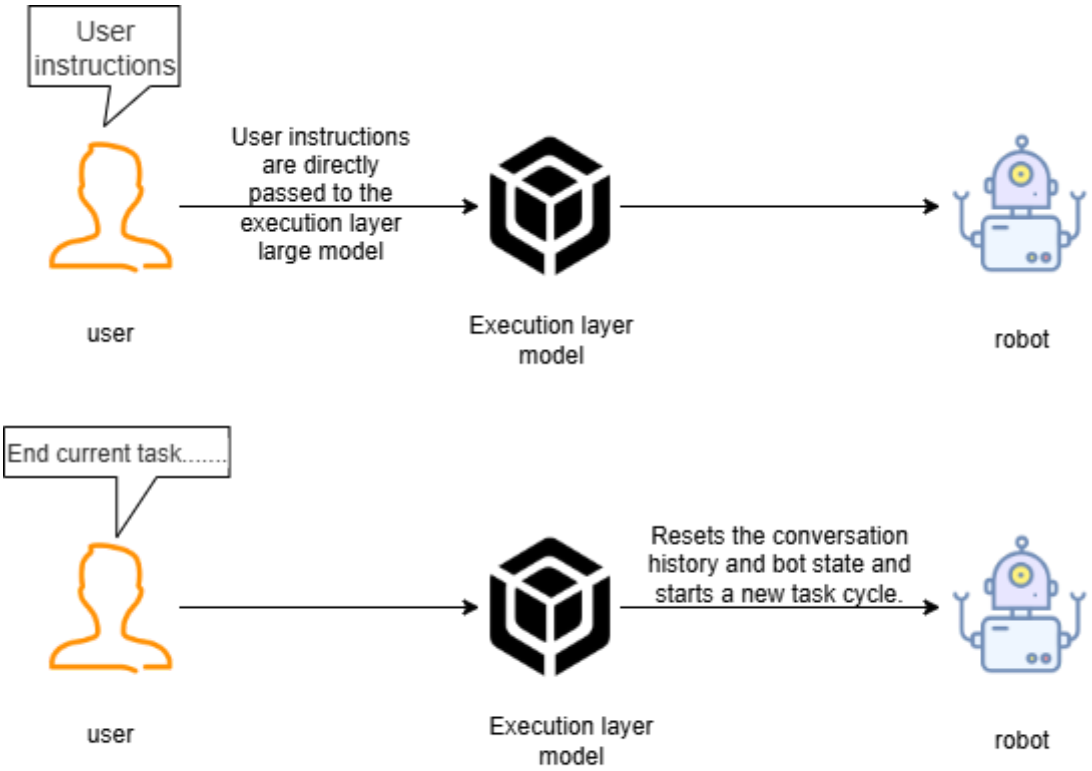
- Query weather
- Calculate math problems
- Guess object English
- Tell jokes

4.4 Execution Layer Large Model

- The robot's execution layer large model uses the Tongyi Qianwen-VL series model by default.
- The execution layer large model is mainly responsible for generating action lists that control the robot's movements and replies to users. During the robot's execution of the action list, it continuously receives feedback on robot action execution results (success/failure) and visual images, and infers the next action to be executed based on whether the action execution state is successful or not
- The execution layer large model acts as a supervisor, continuously supervising the progress of robot task execution steps, and thinking and judging the next action to be executed based on the result feedback and environment information after the robot executes actions, until the task is successfully completed or ended early due to special circumstances.

4.4.1 Task Cycle

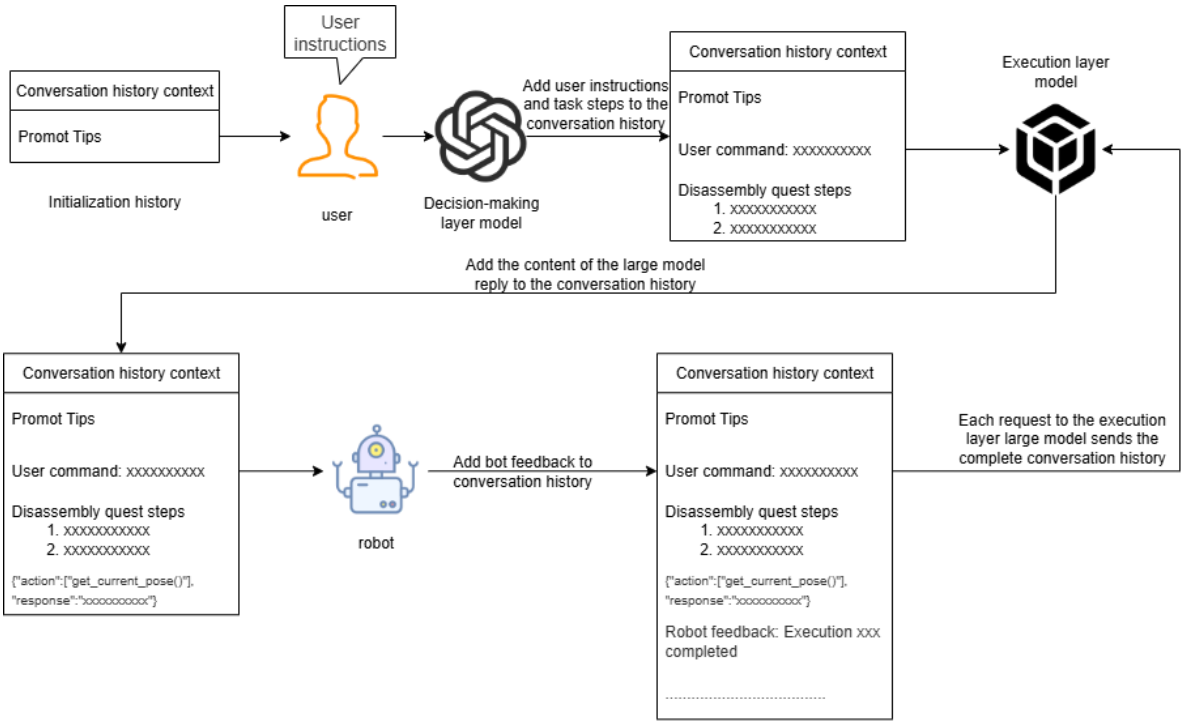
- In a new task cycle, user instructions first go through the decision-making layer large model, then through the execution layer large model. If the execution layer large model determines that the robot has completed all task steps, it will enter **waiting state**. At this time, subsequent user instructions will be directly given to the execution layer large model.
- For example: when the user requests to end the current task and let the robot rest, the robot will reset conversation history and robotic arm state, start a new task cycle, and user instructions will start executing from the decision-making layer large model. The following is a waiting state diagram:



The waiting state of the robot after completing the task

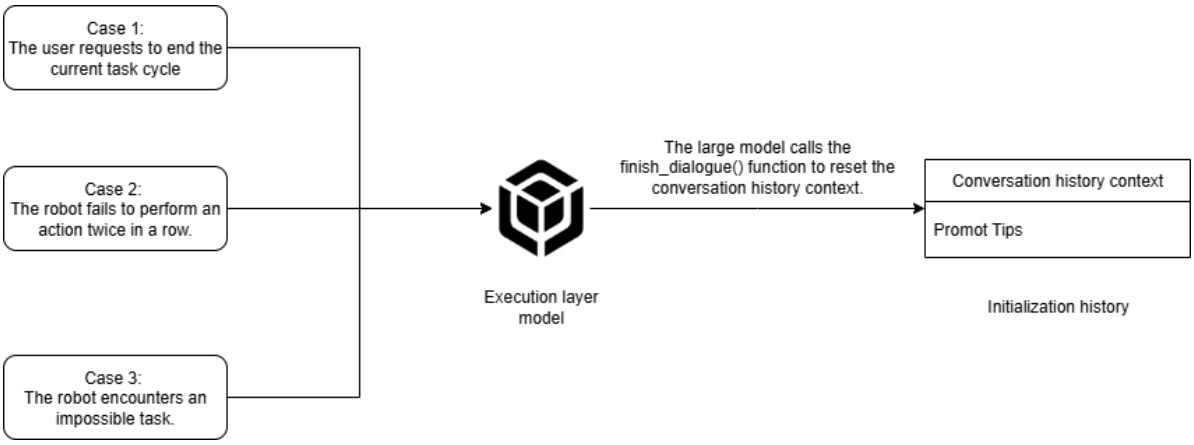
4.4.2 Historical Context

The robot maintains a conversation history context in its local program. The conversation history context contains user instructions, task steps generated by the decision-making layer model, action lists generated by the execution layer model, and robot feedback status information. Each time the robot requests the execution layer large model, it sends the complete conversation history. Therefore, the execution layer large model can know the progress of robot task execution. The execution layer large model will combine the conversation history context to infer the next action that should be executed. In summary, **the execution layer large model will deduce the next future action based on the sum of past states**.



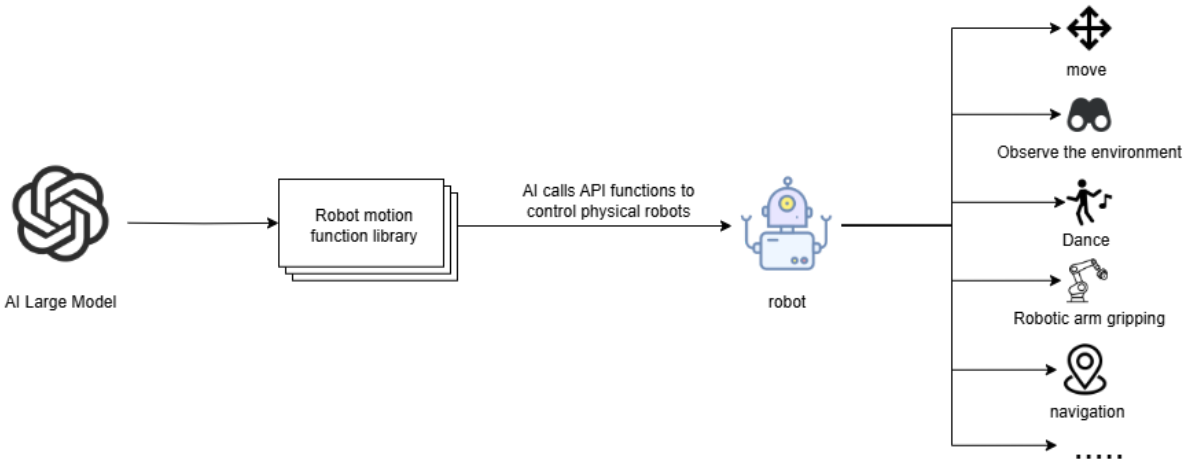
Under normal circumstances, the execution layer large model will determine task completion progress based on conversation history. When it determines that the robot has completed all task steps, it will enter **waiting state**, waiting for user temporary instructions. Temporary instructions will not go through the decision-making layer large model but will be directly passed to the execution layer large model. There are three situations that will cause the robot to clear the conversation context history, end the current task cycle, and start a new task cycle.

- Situation 1: User actively requests to end the task cycle. In **waiting state**, when the user says something like "You can rest now" or "You can step down", the execution layer large model will call the finish_dialogue() function to end the current task cycle, let the robot reset conversation history, and start a new task cycle. The same applies to the following situations.
- Situation 2: The robot fails twice consecutively when executing a certain action. If an action fails, the execution layer large model will control the robot to retry at most once. If it fails again, the execution layer large model will control the robot to end the task cycle early and start a new task cycle.
- Situation 3: When the robot encounters an impossible task, for example, when the user asks the robot to go to a location not mentioned in the "map mapping", the robot will request to end the task cycle early and inform the user that this task cannot be completed.



4.4.4 Robot Action Function Library

The API functions in the robot action function library are the bridge for large models to control the robot's interaction with the real world. These API functions define the minimum actions that physical robots can complete in the physical world. The principle of API functions is to control the underlying hardware of physical robots through the ROS2 system to complete various functions.



All action functions and corresponding functions are shown in the following table:

Function Name	Parameters	Implemented Function	Call Command
arm_dance()	-	Trigger robotic arm dance action	Robotic arm dance
arm_up()	-	Robotic arm upward lift	Robotic arm upward
arm_down()	-	Robotic arm downward movement	Robotic arm downward
arm_nod()	-	Robotic arm nod	Nod
arm_shake()	-	Robotic arm shake head	Shake head
arm_applaud()	-	Robotic arm applaud	Applaud
grip_pose()	-	Robotic arm clamp object posture	Robotic arm adjust to clamp posture
track_pose()	-	Robotic arm track object posture	Robotic arm adjust to track posture
cancel_KCF_follow	-	Cancel tracking object	Cancel tracking object
grasp_obj(x1, y1, x2, y2)	(x1, y1, x2, y2) Top-left and bottom-right point coordinates of target object's outer bounding box	Robotic arm grasp object	Grasp xxx
remove_obj(x1, y1, x2, y2,color)	(x1, y1, x2, y2) are the top-left and bottom-right point coordinates of the outer bounding box of the top color block of obstacle color block. color represents the color of obstacle color block.	Remove obstacle color block	Remove x color block
putdown()	-	Robotic arm put down already clamped item	Put down xxx

Function Name	Parameters	Implemented Function	Call Command
apriltag_sort(x)	x：Machine code number	Sort out specified numbered machine code	Sort out x number machine code
track(x1, y1, x2, y2)	(x1, y1, x2, y2) Top-left and bottom-right point coordinates of target object's outer bounding box	Track specified object in field of view	Track xx
apriltag_remove_higher（x）	x: Height	Remove machine code at specified height	Remove machine code higher than x cm
color_remove_higher（color, target_high）	color：Color target_high：Height	Remove color blocks at specified height	Remove color blocks higher than target_high cm of color color
garbage_sort(type_)	type_: Garbage type to be sorted	Sort specified type of garbage	Sort recyclable garbage
change_pose(self, x1, y1, x2, y2,src,tar,side,tar_size)	x1, y1, x2, y2: Represent the top-left and bottom-right corner coordinates of the top outer bounding box of target placement color block; src represents the color of target transport color block, tar represents the color of target placement color block, side represents placement direction, tar_size represents the height of placement color block	Move color block to direction of another color block	Place blue block on top of yellow block

Function Name	Parameters	Implemented Function	Call Command
record_video(time_)	time_ represents video recording duration	Record video	Record x seconds of video
apriltag_follow(target_id)	target_id: Machine code ID	Track machine code with specified ID	Track x number machine code
compute_pose(x1,y1,x2,y2,name)	x1,y1,x2,y2: Top-left and bottom-right corner coordinates of outer bounding box of color block whose position needs to be recorded, name represents the color of color block whose position needs to be recorded	Record current position of specified color color block	Record current position of x color block
compute_pose_order(x1,y1,x2,y2,name,order)	x1,y1,x2,y2: Top-left and bottom-right corner coordinates of outer bounding box of top color block, name color of top color block, order represents color block stacking order from top to bottom	Record position of each color block in stacked color blocks	Record current position of color blocks on desktop
return_to_orin(name)	name：Represents the color of color block that needs to return to position	Put specified color color block back to original position	Put x color block back to original position
light_on(color)	color：LED light color	Turn on LED light	Turn on * color light
light_off()	-	Turn off LED light	Turn off light
beep_on()	-	Turn on buzzer	Buzzer alarm
beep_off()	-	Turn off buzzer	Turn off buzzer

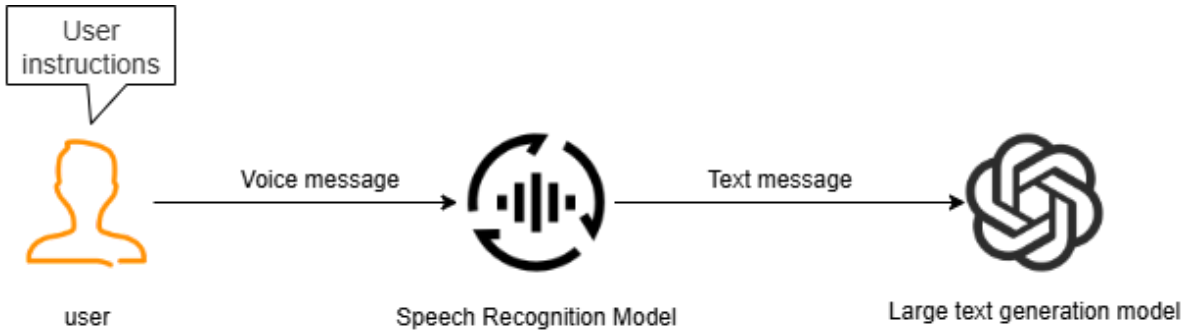
Function Name	Parameters	Implemented Function	Call Command
adjust_joint(joint_id,angle)	joint_id：Servo ID，angle：Servo angle	Set servo angle	Set servo number x to y degrees
gripper_open()	-	Open gripper	Open gripper
gripper_close()	-	Close gripper	Close gripper
color_back_to_orin()	-	Publish color block return to position order	Put color blocks on desktop back to original position
grasp_from_rm_list(color)	color: Color block color	Grasp color block from clamp list	Grasp x color block in clamp list
arm_move(Dir,Dist)	Dir：Direction of robotic arm end adjustment, values are up, down, left, right, front, back, Dist distance of robotic arm end adjustment, unit is centimeters	Adjust position of robotic arm end	Robotic arm adjust x by y cm
point_to(x1, y1, x2, y2)	x1, y1, x2, y2：Top-left and bottom-right corner coordinates of outer bounding box of pointed object	Robotic arm point to object	Robotic arm point to x
garbage_sort_all()	-	Garbage label code sorting	Clean garbage on desktop
grasp_from_down_list(color)	color：Color block color	Grasp color block from placement list	Grasp x color block from placement list
check_remove(color)	-	Check if specified color block exists in remove list	Check if x color block exists in remove list

Function Name	Parameters	Implemented Function	Call Command
seewhat()	-	Take a photo of robot's current perspective and upload to execution layer large model	Observe environment
video_understanding()	-	Understand video content	What is the content in the video just recorded
finish_dialogue()	-	Reset history context, start new task cycle	-
wait(x)	x: Time, unit: seconds	Wait for a period of time, execute no operations	Wait x seconds
finishtask()	-	Automatically called when robot completes task, used to stop feeding back status to execution layer model	-

5. Application of Voice Models in the System

5.1 Voice Recognition

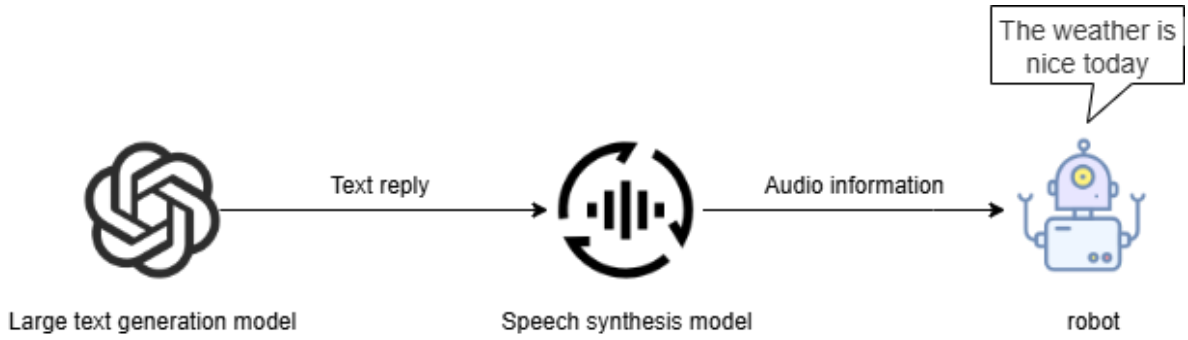
Since the decision-making layer and execution layer large models are text generation models and visual multimodal models respectively, they cannot directly receive user voice information. Therefore, voice recognition large models are needed to convert user voice commands into corresponding text before handing them to AI large models.



5.2 VAD Voice Activity Detection

VAD (Voice Activity Detection) is a technology that automatically identifies speech segments and non-speech segments (such as silence, noise) in speech signals. It locates the start and end positions of speech from audio streams and filters out invalid background sounds.

In subsequent practical courses, when the user uses the wake-up word "Hello Xiaoya" to wake up the robot, it will enter VAD voice activity detection. The system will automatically detect the user's speaking duration and save effective sound segments as WAV audio files, then give the audio files to the voice recognition model to convert into text.

5.3 Voice Synthesis

The text replies generated by the large model for users are converted into audio through voice synthesis models, and then played by hardware speakers.